TWIN-64
Architecture Document

Helmut Fieres
July 28, 2025

Contents

Introduction

Designers of the seventies and early eighties CPUs almost all used a micro-coded approach with hundreds of instructions. RISC was just starting to enter the stage. Registers were typically not really generic but often had a special function or meaning and many instructions were rather complicated and the designers felt they were useful. The compiler writers often used a small subset ignoring these complex instructions because they were so specialized. The later eighties and nineties shifted to RISC based designs with large sets of registers, fixed word instruction length, a large virtual memory address range and instructions that are in general much more pipeline friendly. What if some of these principles had found their way earlier into CPU designs?

Welcome to Twin-64. Twin-64 is a simple 64-bit CPU with a register-memory model and segmented virtual memory. The design is heavily influenced by Hewlett Packard's PA_RISC architecture, which was initially a 32 bit RISC-style register-register load-store machine. Almost all of the key architecture features come from there. However, other processors, such as the Data General MV8000, the DEC Alpha, and the IBM PowerPc, were also influential or at least investigated for their architectural design choices. The original design goal that started this work was to truly understand the design process and implementation tradeoffs for a simple pipelined CPU.

While it is not a design goal to build a modern, competitive CPU, the CPU should nevertheless be useful and largely follow established common design practices. For example, instructions should not be invented because they seem to be useful, but rather designed with the assembler and compilers in mind. Instruction set design is also CPU design. Register memory architectures were typically micro-coded complex instruction machines. In contrast, VCPU-32 instructions will be hard coded and are in general "pipeline friendly", avoiding data and control hazards and stalls where possible.

Concepts

This section outlines the guiding principles and concepts chosen for Twin-64.

The instruction set design guidelines center around the following principles. First, in the interest of a simple and efficient instruction decoding step, instructions are of fixed length. A machine word is the instruction word length. As a consequence, some address offsets are rather short and addressing modes are required for reaching the entire address range. Instead of a multitude of addressing modes, typically found in the CISC style CPUs, Twin-64 offers very few addressing modes with a simple base address - offset calculation model. No indirection or any addressing mode that would require to read a memory item for address calculation is part of the architecture.

There will be few instructions in total, however, depending on the instruction several options for the instruction execution are encoded to make an instruction more versatile. For example, a boolean instruction will offer options to negate the result thus offering and "AND" and a "NAND" with one instruction. Wherever possible, useful instruction options such as the one outlined before are added to an instruction, such that it covers a range of instructions typically

found on the CPUs looked into. Great emphasis is placed in that such options do not overly complicated the pipeline and only increase the overall data path length slightly.

Modern RISC CPUs are typically load/store CPUs and feature a large number of general registers. Operations takes place between registers. Twin-64 follows a slightly different route. There is a smaller number of general registers and in addition to computation between registers, a register-memory model is also supported. There are however no instructions that read and write to memory in one instruction cycle as this would complicate the design considerably.

Twin-64 offers a large address range, organized in segments. Segment Id and offset from a virtual address. In addition, a short form of a virtual address, called a logical address, will select segment register based on the upper two bits of the logical address to form a virtual address. Segments are also associated with access rights and protection identifies. The CPU itself can run in user and privilege mode.

This document describes the overall architecture, instruction set and runtime model for Twin-64. It is organized into several parts. The first part will give an overview on the architecture. It presents the memory model, registers sets and basic operation modes. The major part of the document then presents the instruction set. Memory reference instructions, immediate instructions, branch instructions, computational instructions and system control instructions are described in detail. These chapters are the authoritative reference of the instruction set.

The runtime environment chapters present the runtime architecture. Although the CPU architecture and instructions allow for a great flexibility how to write programs, a runtime architecture is essential to define the common usage of the instruction set for assembler and compilers.

This book

This book defines the overall Twin-64 architecture. It is organized into several chapters.

Architecture Overview

64 bit architecture

multi processor

processors have a TLB and a cache

52 bit virtual memory

34 bit physical memory

I/O is memory mapped

I/O Modules - System Bus

instructions feature register memory and load / store concept

register offset and register indexed, scalable

A Register Memory Architecture

Twin-64 implements a **register memory architecture** offering few addressing modes for the operand combined with a fixed instruction word length. For most instructions one operand is the register, the other is a flexible operand which could be an immediate, a register content or a memory address from where the data is obtained or placed. The result is placed in the register which is also the first operand. These type of machines are also called two address machines.

In contrast to similar historical register-memory designs, there is no operation which does a read and write operation to memory in the same instruction. For example, there is no "increment memory" instruction, since this would require two memory operations and is in general not pipeline friendly. Memory data access is performed on a machine word, half-word or byte basis. Besides the implicit operand fetch in a computational instruction, there are dedicated memory load / store instructions. In addition to the register-immediate operand and register memory-operand mode, one operand mode supports the three operand model (Rs = Ra OP Rb), specifying two registers and a result register, which allows to perform three address register for computational operations as well. The machine can therefore operate in a memory register model as well as a load / store register model.

Signed arithmetic

only signed 64 bit math

Virtual Memory

System Organization

Data Types

Twin-64 is a big endian 64-bit machine. The fundamental data type is a 64-bit signed integer with the most significant bit being left. Bits are numbered from left to right, starting with bit 64 as the MSB bit down to bit 0 as LSB. Memory access is performed on a double, word, half-word or byte basis. All addresses are however always expressed as bytes addresses.

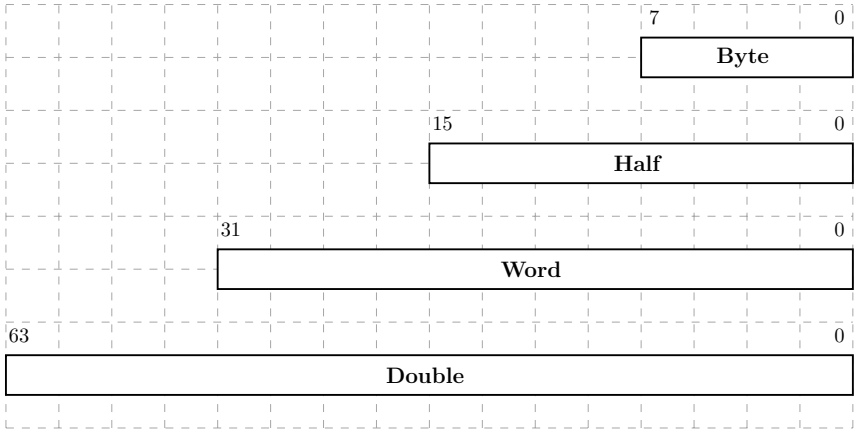


Figure 1: Data types

lower sizes are sign extended before use

address computation uses a 32-bit unsigned math, i.e. numbers wrap around in 32 bit space.

Byte Ordering

Twin-64 supports little endian and big endian byte ordering models.

General Registers

Twin-64 features a set of 16 general purposes registers.

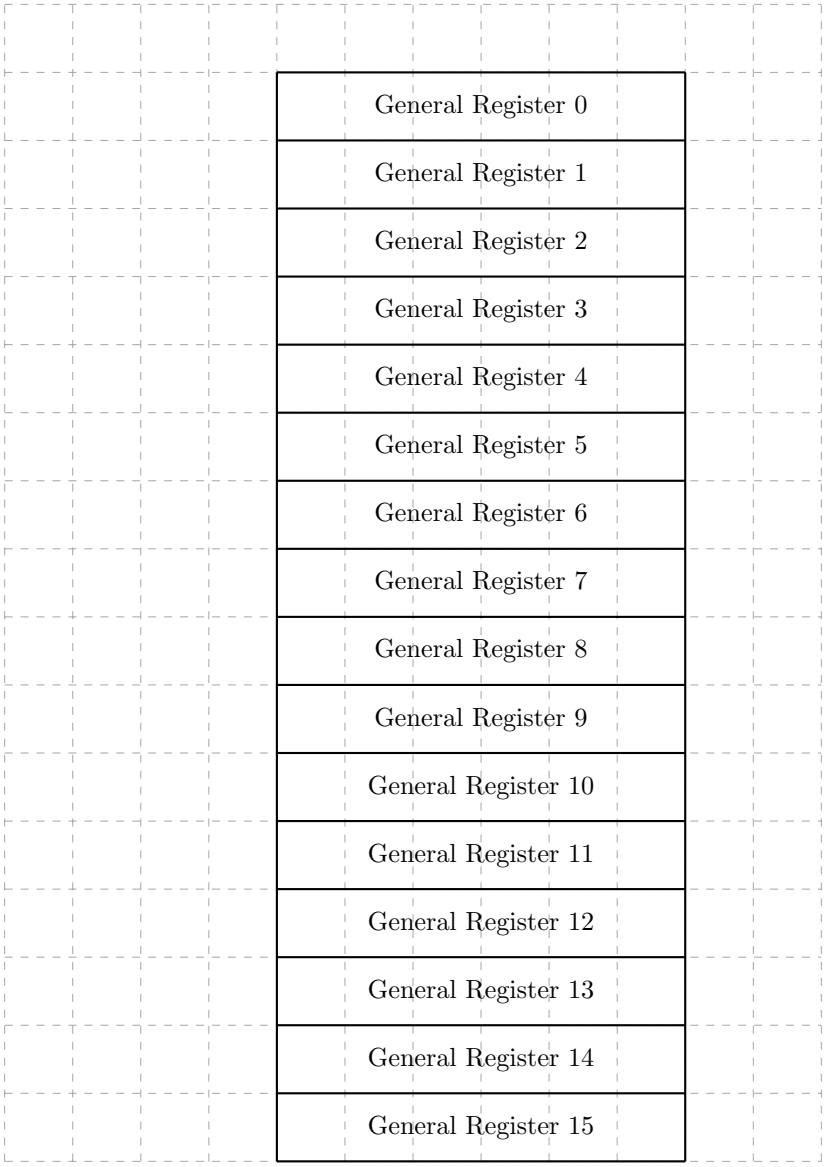


Figure 2: General registers

Control Registers

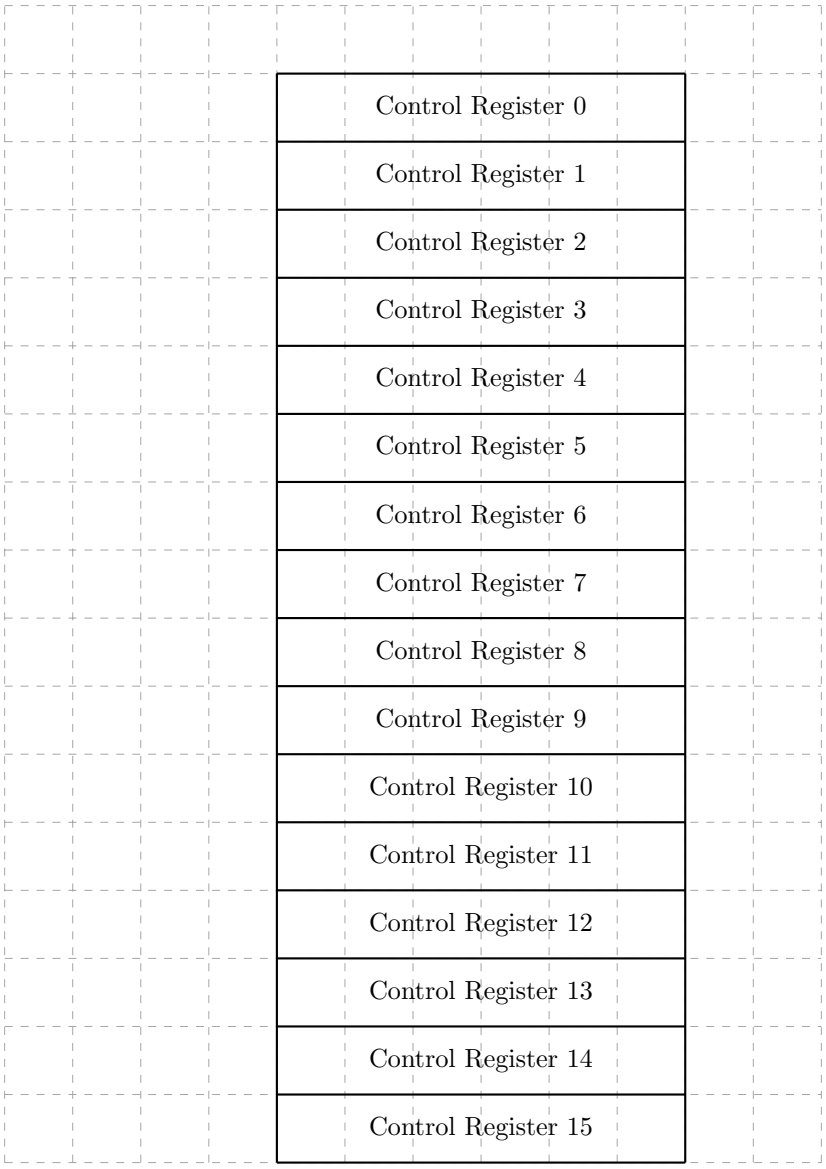


Figure 3: General registers

Some general registers have a dedicated use. Register zero will always return a zero value when read and a write operation will have no effect. Although there are only 16 general registers in the CPU, implementing such a register greatly simplifies the instruction design, in that it for example provides a target when the result is not needed. General register one is a scratch register and also serves as an implicit target for some instructions. Other general registers may have dedicated purpose defined in the runtime architecture.

Program State Register

Figure 4: Program state register

Table 1: Status Field Bits

Head 1	**Head 2**
text 1	text
text 2	text

TLB

For performance reasons, the virtual address translation result is stored in a translation look-aside buffer. The TLB is essentially a copy of the page table entry information that represents the virtual page currently loaded at the physical address. Depending on the hardware implementation, there can be a combined TLB or a separate instruction TLB and data TLB. A TLB is essentially a cache for address translations. The virtual page number, VPN, is stored along with attributes and the physical page number, PPN, in the TLB hardware. The current architecture allows for a 34-bit physical address space, which is a 16GB memory space.

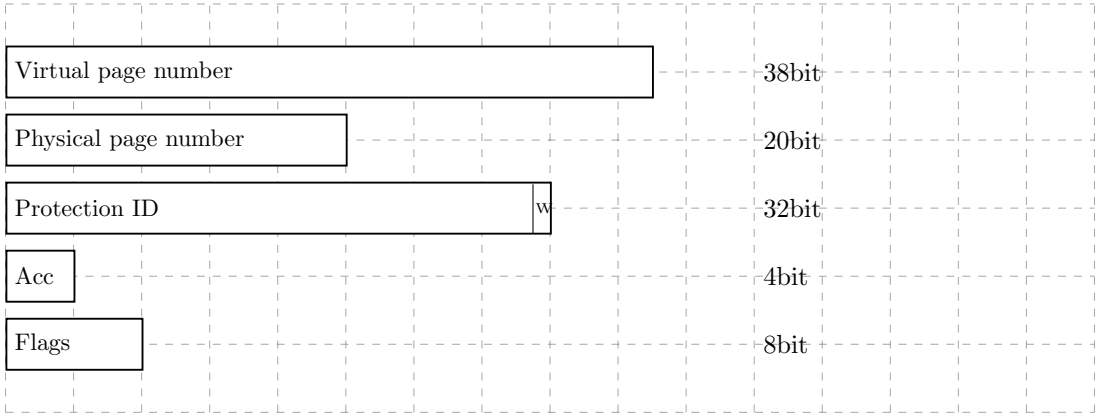


Figure 5: Tlb info

The first four segments, segment 0 to 3, are special in that they bypass the TLB entirely. The address range represents physical memory and can only be access in privileged mode. A virtual address in the range 0x00_0000_0000 to 0x03_FFFF_FFFF directly maps to the physical address of the same range.

To reduce TLB processing overhead, the TLB is capable of storing virtual address ranges. There is support for a large page size of 16Kb, 256 Kb, 4Mb and 64 Mb.

38bit VPN, 20bit PPN, 4bit AR, 32 bit PID, 2bit size, xbit flags

bits:

V - valid

L - entry locked

M - modified trap

B - branch taken trap

U - uncacheable

Table 2: TLB Fields

Head 1	**Head 2**
text 1	text
text 2	text

Caches

Twin-64 is a cache visible architecture, featuring cache models for unified or separate instruction and data cache. While hardware directly controls the cache for cache line insertion and replacement, the software has explicit control on flushing and purging cache line entries. There are instructions to flush and purge cache lines. To speed up the entire memory access process, the cache uses the virtual address to index into the arrays, which can be done in parallel to the TLB lookup. When the physical address tag in the cache and the physical page address in the TLB match, there will be a cache hit. The architecture will not specify the size of cache lines or cache organization. Typically a cache line will hold four to eight machine words, the cache organization is in the simplest case a single set direct mapped cache. The Twin-64 architecture does not specify a particular cache and cache hierarchy model. The architecture just specifies instructions to handle cache operations, such as delete or flushing a cache.

System Memory

Virtual Address Space

Twin-64 features a 52-bit virtual address range. The address range is organized into a 20-bit segment identifier and a 32-bit offset into that segment. The following figure shows the virtual address and how the segment is selected.

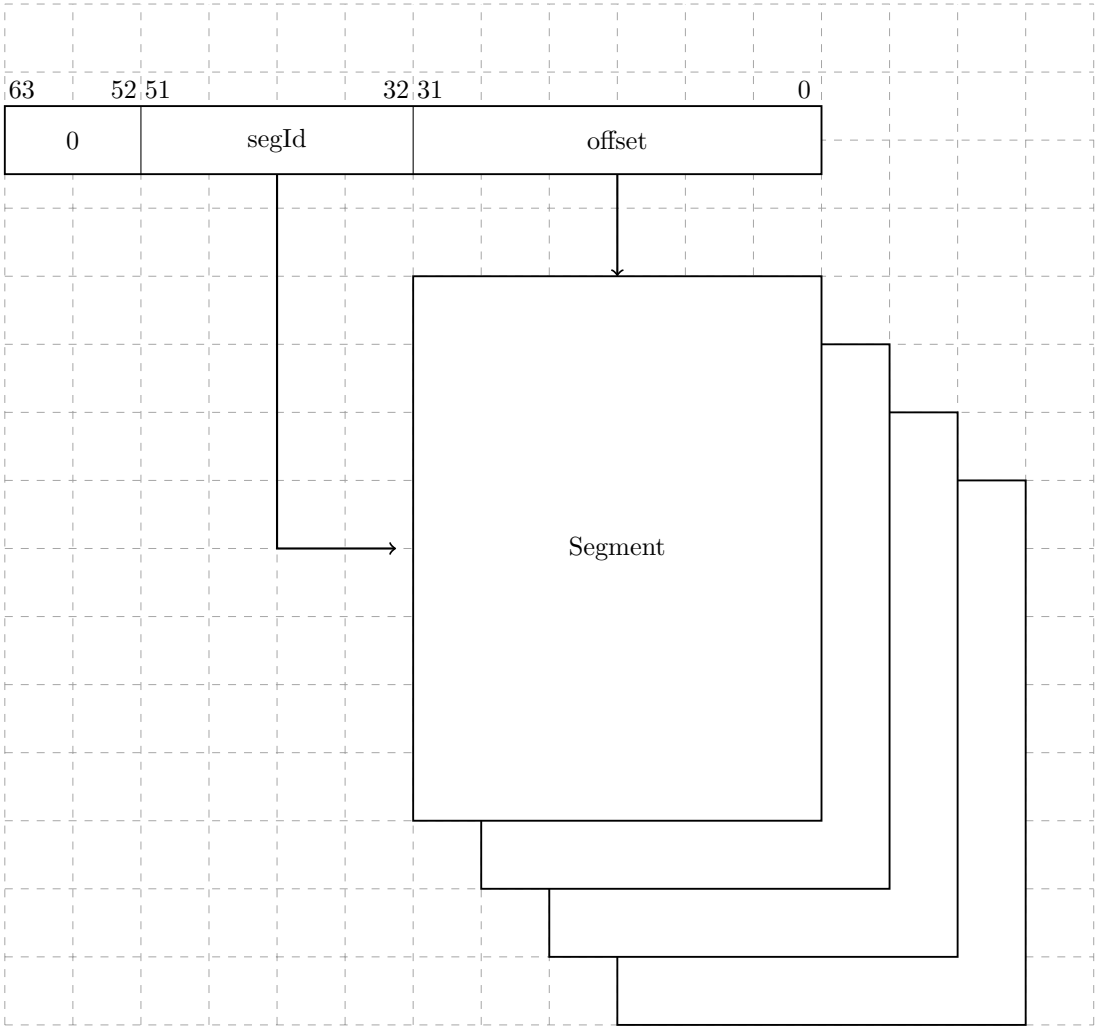


Figure 6: Virtual Memory Address Range

A virtual address to be translated to a physical address. For address translation purposes, the virtual address range can also be seen as a set of virtual pages. The architected page size is 16Kb. A virtual page number is a 38-bit page number with a 14-bit offset into that page.

Physical Address Space

Twin-64 architecture features a 34-bit physical memory address range which allows a maximum physical memory of up to 16 GBytes.

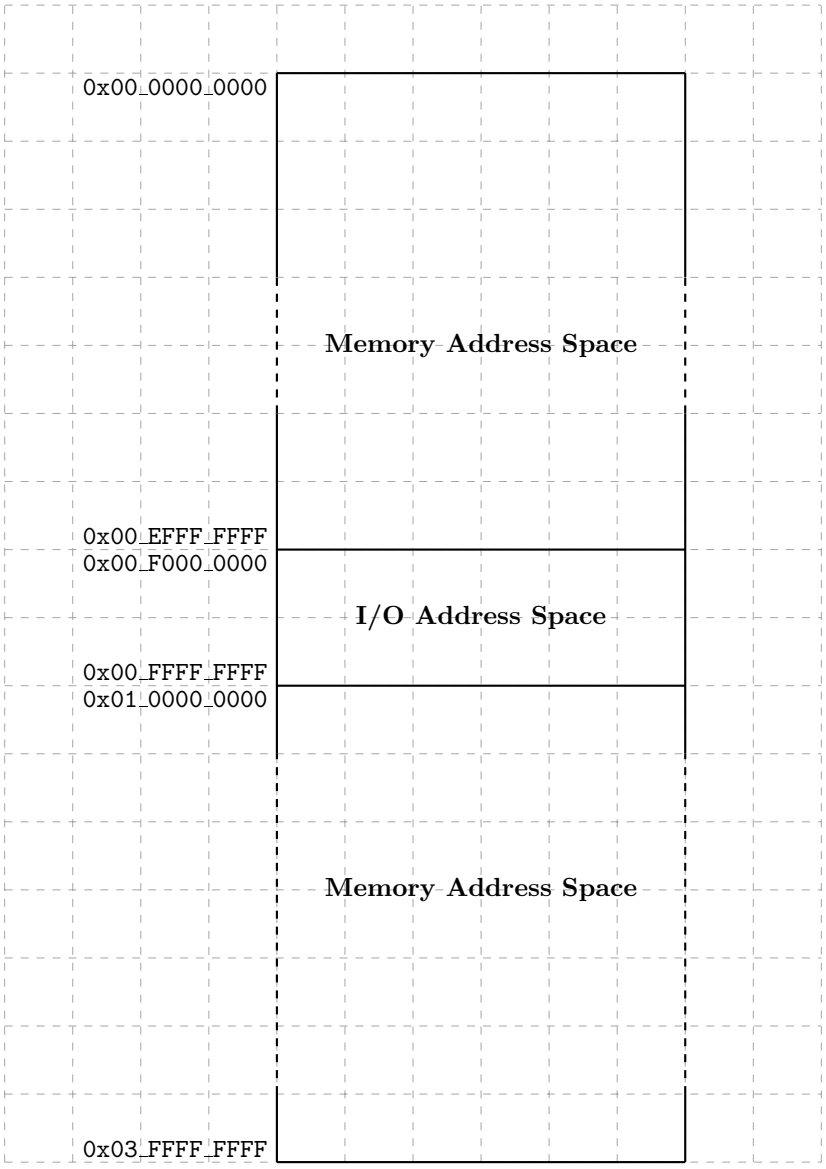


Figure 7: Physical Memory Address Range

segment Id 0 to 3 directly map to the physical address, without TLB translation and access rights checking. Only privileged mode can access these segments.

I/O Address Space

Twin-64 features memory mapped I/O architecture. As shown in the overall physical memory layout, the physical memory address range is located from `0x00_F000_0000` to `0x00_FFFF_FFFF`. The following figure depicts the I/O memory address space ranges in more detail.s

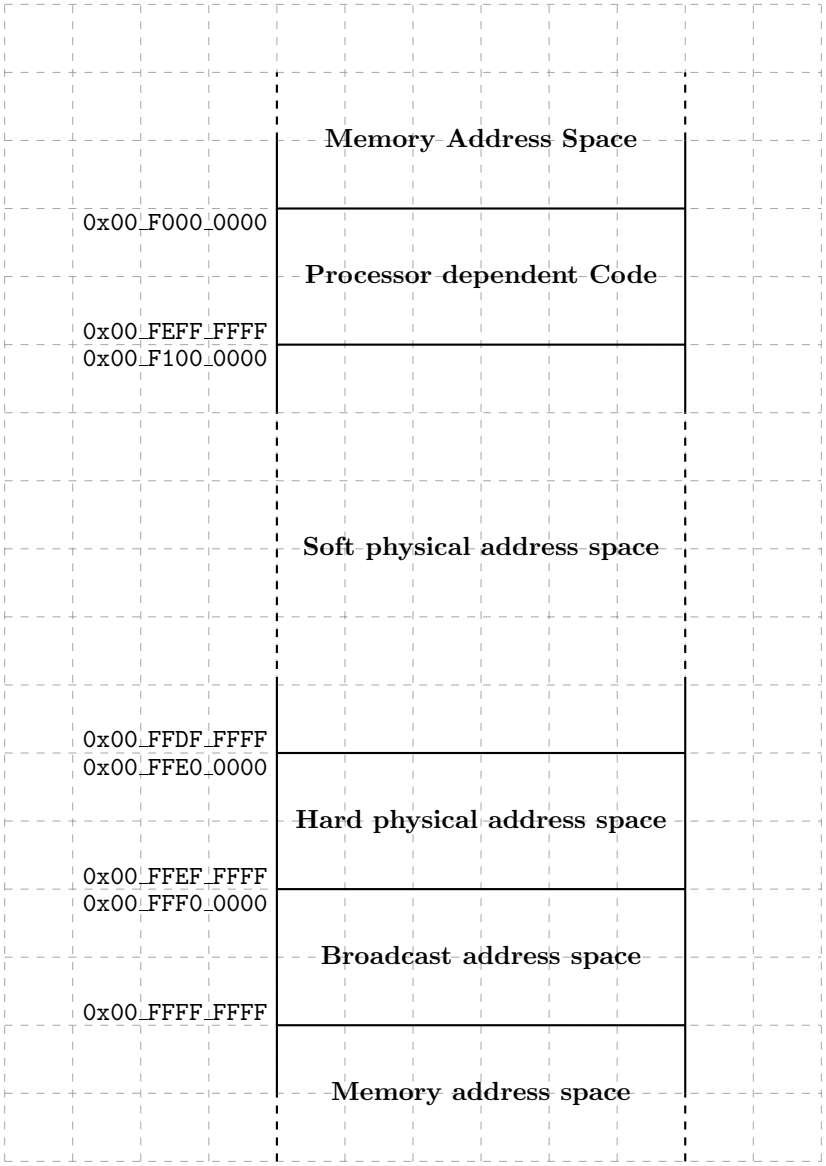


Figure 8: I/O Memory Address Range

always uncached

The I/O address space dedicates 256 Mbytes to an area that contains the processor dependent code. Typically, this is a set of library functions to manage the processor itself but also provide low-level primitives for the boot process and the operating system. The remainder of the I/O address space is divided into I/O modules, which map the I/O hardware components to a memory addressable location. Both PDC and I/O firmware design is explained in a later part of the document.

Addressing and Access Control

Address Translation

The Twin-64 CPU emits always a virtual address. Each memory access needs to be translated into a physical address.

virtual address for the first four segments directly map to the physical address space.

Twin-64 implements a protection model along two dimensions.

Access Control

The first dimension is the **access control** and **privilege level** check. Each page is associated with a page type. There are read-only, read-write, execute and gateway pages. Each memory access is checked to be compatible with the allowed type of access set in the page descriptor. There are two privilege levels, user and supervisor mode. Access type and privilege level form the access control information field, which is checked for each instruction and data memory access. For read access the privilege level us be least as high as the PL1 field, for write access the privilege level must be as least as high as PL2. An exception are the execute and gateway pages, where a write access is only allowed for privilege code. For execute access the privilege level must be at least as high as PL1 and not higher than PL2. If the instruction privilege level is not within this bounds, a privilege violation trap is raised. If the page type does not match the instruction access type an access violation trap is raised. In both cases the instruction is aborted and a memory protection trap is raised.

Access Protection

The second dimension of protection is a **protection ID**. Twon-64 allows to record a set of segment Id values in the processor control registers. For each access to a segment that has the protection check bit set, one of the protection Id control registers must match the segment Id. If not, a protection violation trap is raised.

Protection Id checking is typically used to form a grouping of access. A good example is the stack data segment, which is accessible at user privilege level. Since the CPU features a global virtual memory address space, every task could access a data stack of another task. With protection Id checking enabled and the segment Id loaded in one of the control registers, only the corresponding user task is allowed to access the stack data segment with a matching protection Id.

Interruptions

Traps

Table 3: Traps

Head 1	Head 2
text 1	text
text 2	text

External Interrupts

Input/Output

Concepts

memory mapped

dedicated address space in physical memory

bus concept

module concept

modules have registers - load / store, however registers do not have to be 64 bit wide.

the system bus features up to 64 modules. that is: 64 HPA pages, need: 1Mb address range.

a module listens to three address ranges

- hard physical address range. a page. privileged.

- soft physical address range. allocated by the module for further space. aligned on a page boundary.

- broadcast address range. Mirrors the system bus range. A write to a register in that range applies to all corresponding registers of a module on the same bus.

modules are processors, memory and I/O cards.

we could keep processor stats in the HPA space

memory module has entire memory as SPA space

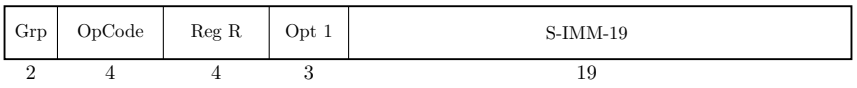
Instruction Set

Instruction Overview

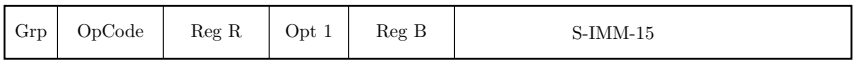
This chapter presents the Twin-64 instruction set. Like typical RISC style instruction sets, it is a fixed length instruction set with very few regular formats. The instructions are group into four categories, arithmetic and logical, load and store, branch and special and system instructions.

Instruction Formats

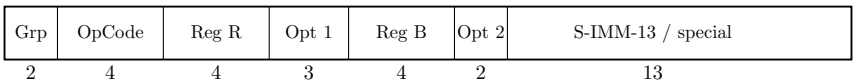
Twin-64 uses few instruction formats. They are grouped by the number of available register slots and the length of the immediate value field. The **signed immediate S19** instruction format provides one register and a 19 bit signed immediate field. This format is typically used by branch instructions.



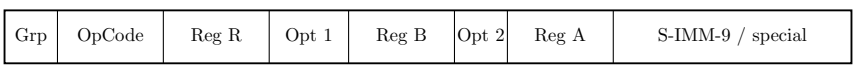
The **signed immediate S15** instruction format provides two registers and a 15 bit signed immediate field. This format is used by branch instructions as well as instruction which operate on a register and an immediate value.



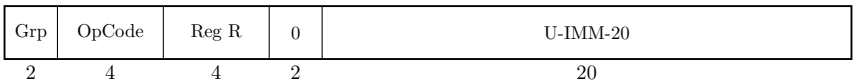
The **signed immediate S13/special** instruction format provides two registers, an additional option field and a 13 bit signed immediate field. Some instruction use the S-IMM-13 field for special encodings instead of a signed value. This format is used by memory reference instructions where the address is formed by a base register and a 13-bit signed offset.



The **register/signed immediate S9/special** instruction format provides three registers, an additional option field and a 9 bit signed immediate field. Some instruction use the S-IMM-9 field for special encodings instead of a signed value.instead of a signed value. This instruction format is used for all computational instructions where three registers are needed.



The **unsigned immediate U20** format is used by the immediate instruction group to provide a 20bit unsigned value. This value is then placed at certain positions in the register target Reg R.



Arithmetic Instructions

The arithmetic instruction operate on two signed 64-bit operands and stores the result to the target register. There are two instruction layouts, signed immediate S15 and register based. The **immediate operand instruction format** will add the sign extended 15-bit value to `RegB`. The **register instruction format**type instruction uses two registers as input operands, performs the operation and stores the result in a third register. The following figure shows the instruction layout.

The **ADD** and **SUB** instructions perform signed integer 64-bit arithmetic with a numeric overflow resulting in an overflow trap. There are no operations on unsigned quantities. The **SHLxA** and **SHRxA** perform an arithmetic left shift operation and add an operand to the shifted input, storing the result in the target register.

The **CMP** compares two registers for being equal, not equal, less than and les or equal than. The result of this comparison is store din the target register.

Bit Operations Instructions

The bit operation instruction fall into two groups. The **AND**, **OR** and **XOR** instructions implement the logical operations as their name suggest. Like the arithmetic instructions, there is an immediate instruction and a register instruction layout.

The second group implements the bit manipulation operations. The **EXTR** instruction extracts a bit field from a register and places the field in another register. Optionally, the extracted field is signed extended. The **DEP** instruction deposits a bit field into a target register.

The **DSR** instruction concatenates two general registers, shifts the 128-bit quantity to the right and stores the lower 64 bits in a target register.

Immediate Instructions

The **LDI**, and **ADDIL** are instruction for forming large constants. With a 32-bit instruction word, even 32-bit constants are not possible in one instruction. The **LDI** instruction has a qualifier to load a constant value at a defined position in the target register. The opt field allow to select fields in the target register where the constant value is stored. The **ADDIL** instruction adds the upper portion of a 32-bit immediate value to the lower 32-bit half of a general register. This instruction is used primarily for address arithmetic to allow reaching any location in a segment.

The **LDO** instruction loads an offset into a general register. The address loaded is formed by adding the signed immediate value to the base register.

Memory Reference Instructions

Twin-64 is a register memory architecture. It has the common load/store instructions as well as instructions that combined an operation with a memory reference. Memory reference instructions feature two addressing modes. The first is the base register and offset instruction. the second the base register and register index instruction.

The **indexed instruction format** will load the content of memory address formed by adding the scaled immediate 13-bit field to the base register `RegB` and add the data value to `RegA`. The **dw** field indicates the data value width. A **dw** value of zero load an 8-bit, a value of 1 a 16-bit value, value of 2 a 32-bit value and a value of 3 a 64-bit value. In any case, the data value fetched is signed extended to a 64-bit value. In addition, the **dw** field will scale the offset value by left shifting the immediate field according to the data width. The following figure shows the general instruction layout for base register and offset instruction.

The **register indexed instruction format** will load the content of memory address formed by adding `RegX` to the base register `RegB` and add the data value to `RegA`. Both load formats will use the **dw** field to specify the data value width. A **dw** value of zero load an 8-bit, a value of 1 a 16-bit value, value of 2 a 32-bit value and a value of 3 a 64-bit value. In any case, the data value fetched is signed extended to a 64-bit value. The offset in `RegX` is interpreted as a byte offset, independent of the actual **dw** field value. The following figure shows the general instruction layout for base register and register index type instructions.

The **LD** and **ST** instruction are the load and store instructions. In addition to the two addressing modes presented above, they also feature a base register address modification. Depending on the offset sign, the base register is updated before or after the address computation with the effective address computed. See the instruction description for more details.

The **LDR** and **STC** instruction implement the base support for a pipeline friendly method for semaphore operations. They only support the base register and offset mode. Also, there is no base register modification option.

The **ADD**, **SUB**, **AND**, **OR**, **XOR**, **CMP** instructions perform the respective operation as described above with one operand being fetched from memory.

Branch Instructions

Control flow is implemented through a set of branch instructions. They can be classified into unconditional and conditional instruction branches.

Unconditional branches fetch the next instruction address from the computed branch target. The **B** instruction performs an instruction relative branch, the **BR** instruction a base register relative branch. Both optionally return the return address in a general register.

The **BV** instruction is a vectored branch. The content of a general register is added to a base register, forming branch target.

The instructions **BB**, **CBR** and **MBR** implement the conditional branch instructions. If the condition is met a branch to the target address is performed. The target address is always a local address and the offset is instruction address relative. Conditional branches adopt a static prediction model. Forward branches are assumed not taken, backward branches are assumed taken.

All branch address arithmetic is a 32-bit unsigned arithmetic with wraparound. Adding branch offsets will not overflow into the segment Id portion of the address..

System Instructions

The final instruction group is the **special/system** instruction group. The system control instructions are intended for the runtime designer to control the CPU resources such as TLB, Cache and so on. There are instruction to load a segment or control registers as well as instructions to store them. The TLB and the cache modules are controlled by instructions to insert and remove data in these two entities. Finally, the interrupt and exception system needs a place to store the address and instruction that was interrupted as well as a set of shadow registers to start processing an interrupt or exception. Most of the instructions found in this group require that the processor runs in privileged mode.

The **MFCR**and **MTCR** instructions are used to transfer data to and from a control register. Access to some control registers requires privileged mode. The **RSM**and **SSM** instructions allow for setting or clearing an individual an accessible bit of the status part of the processor state.

The **LDPA**, **PRBR** and **PRBW** instructions are used to obtain information about the virtual memory system. The LDPA instruction returns the physical address of the virtual page, if present in memory. The PRB instruction tests whether the desired access mode to an address is allowed.

The TLB and Caches are managed by the **ITLB**, **PTLB**, **FCA** and **PCA** instruction. The ITLB and PTLB instructions insert into the TLB and remove translations from the TLB. The FCA and PCA instructions manages the instruction and data cache, featuring to flush and / or just purge a cache line. There is no instruction that inserts data into a cache as caching is completely controlled by hardware.

The **RFI** instruction is used to restore a processor state from the control registers holding the interrupted instruction address, the instruction itself and the processor status word. Optionally, general registers that have been stored into the reserved control registers will be restored.

The **DIAG** instruction is a control instructions to issue hardware specific implementation commands. Example is the memory barrier command, which ensures that all memory access operations are completed after the execution of this DIAG instruction. The **BRK** instruction is transferring control to the breakpoint trap handler.

ADD Integer Addition

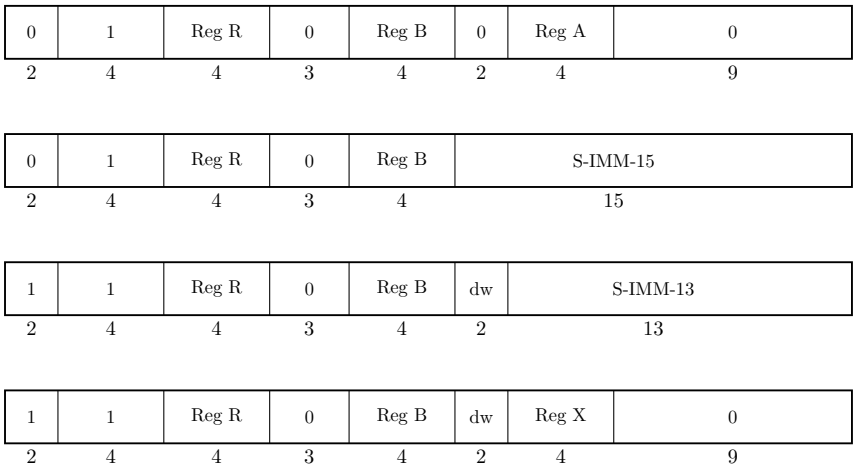
Syntax

```
ADD RegR, RegB, RegA
ADD RegR, RegB, SignedImmed15

ADD[.D/W/H/B] RegR, SignedImmed13( RegB )
ADD.D/W/H/B] RegR, RegX ( RegB )
```

Format

The ADD instruction uses the register, the immediate, the indexed and the register indexed instruction formats.



Description

The `ADD` instruction adds two signed 64-bit operands and stores them to the target register `RegR`. The instruction supports the immediate, the register and the two memory reference instruction formats. An arithmetic overflow results in a numeric overflow trap. The indexed formats may cause a memory reference associated trap.

Operation

`RegR <- RegB + RegA` (register format)
`RegR <- RegB + immOperand( )` (immediate format)
`RegR <- RegR + memOperand( )` (indexed formats)

Exceptions

Integer Overflow
Data Alignment
Memory Access Violation
Memory Protection Violation
Data TLB miss

Notes

None.

SUB Integer Subtraction

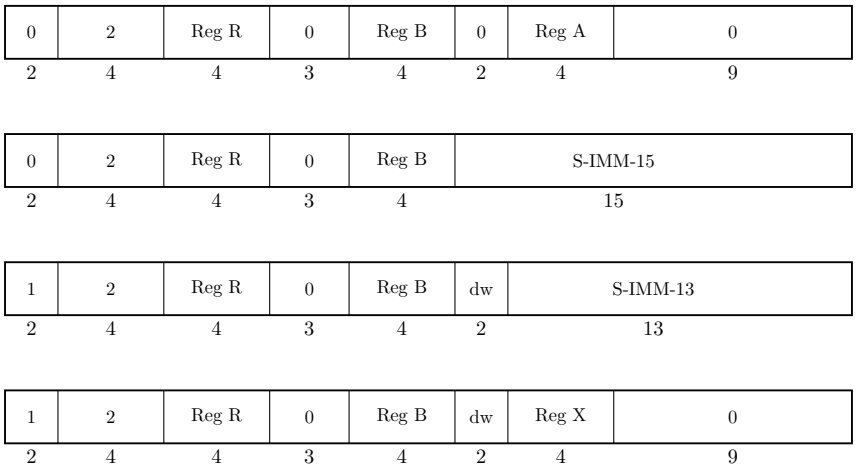
Syntax

```
SUB RegR, RegB, RegA
SUB RegR, RegB, SignedImmed15

SUB[.D/W/H/B] RegR, SignedImmed13( RegB )
SUB.D/W/H/B] RegR, RegX ( RegB )
```

Format

The SUB instruction uses the register, the immediate, the indexed and the register indexed instruction formats.



Description

The `SUB` instruction subtracts a signed 64-bit operand from a general register `RegB` and stores the result to the target register `RegR`. The instruction supports the immediate, the register and the two memory reference instruction formats. An arithmetic overflow results in a numeric overflow trap. The indexed formats may cause a memory reference associated trap.

Operation

`RegR <- RegB - RegA` (register format)
`RegR <- RegB - immOperand( )` (immediate format)
`RegR <- RegR - memOperand( )` (indexed formats)

Exceptions

Integer Overflow
Data Alignment
Memory Access Violation
Memory Protection Violation
Data TLB miss

Notes

None.